Permissions – Practice Class Work

Student: Ramazonov Norbek SE16054 Group: SE1-24 Date: 29.10.2026

Question 1

- **Four decisions in order:** 1. Decide if a permission is needed; 2. Determine the permission category; 3. Check if it's already granted; 4. Handle the denial or request outcome.
- **Example without permission:** Accessing the app's internal files directory (context.filesDir).
- **Why it's best:** Requires zero permissions, keeps user data private, and avoids prompts.

Question 2

- **Three categories & granting:**
  1. **Install-time:** Granted automatically upon app installation.
  2. **Runtime:** Granted by the user via an in-app system dialog.
  3. **Special:** Granted manually on a dedicated system Settings screen.
- **Code complexity:** Install-time needs only a manifest line. Runtime requires state checks, rationale handling, and results. Special requires opening external settings.

Question 3

- **Why normal permissions need no check:** They pose minimal risk, so the system grants them automatically at install.
- **Exception thrown:** Omitting them from the manifest throws a SecurityException.
- **Three examples:** INTERNET, ACCESS_NETWORK_STATE, WAKE_LOCK.

Question 4

- **Signature permissions:** Granted only to apps signed with the exact same digital certificate as the declaring app.
- **Protection level:** protectionLevel="signature".
- **Result & Dialog:** Wallet (same signature) gets access; stranger apps are denied. No dialog is shown to the user.

Question 5

- **When/where shown & granted:** Shown in the Play Store/installer before install, and granted automatically upon installation.
- **Two kinds:** Normal permissions and feature-dependent hardware permissions.

Question 6

- **Version introduced: API 23** (Android 6.0).
- **API 22 behavior:** Runtime-type permissions are granted automatically at install because older apps cannot handle runtime checks.

- **granted(kind, sdk) model:** SDKs below 23 fall back to treat runtime permissions as install-time permissions.

Question 7

- **States in order:** Not asked yet → Denied once → Denied twice ("Don't ask again").
- **shouldShowRationale & code:** True/false on first/second denials to show explanations; becomes false after two denials.
- **Where to send user:** Send the user directly to the app's Settings page (Settings.ACTION_APPLICATION_DETAILS_SETTINGS).

Question 8

- **Three branches of rememberPermissionState:**
    1. isGranted: Shows the core feature UI.
    2. shouldShowRationale: Shows an educational explanation.
    3. Remaining state: Triggers the request or guides to Settings.
- **Surviving recomposition:** It uses Jetpack Compose state mechanisms (remember).
- **ExperimentalPermissionsApi:** Marks the API as experimental and subject to change.

Question 9

- **Without a library:** Use rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission()).
- **Checks:** Read state with ContextCompat.checkSelfPermission; check rationale with ActivityCompat.shouldShowRequestPermissionRationale.
- **Comparison:** Accompanist wraps this raw boilerplate into clean, reactive Compose states.

Question 10

- **RequestMultiplePermissions return:** Returns a Map<String, Boolean> mapping permissions to true or false.
- **Single interface:** The system groups precise and approximate location to reduce user prompt fatigue.
- **If denied both:** Disable location features gracefully and explain why access is helpful.

Question 11

- **Special vs runtime:** Cannot be requested via standard in-app popups; users toggle them manually in system settings.
- **Check table:**
    - SYSTEM_ALERT_WINDOW: Settings.canDrawOverlays(context)
    - WRITE_SETTINGS: Settings.System.canWrite(context)
    - MANAGE_EXTERNAL_STORAGE: Environment.isExternalStorageManager()
    - REQUEST_INSTALL_PACKAGES: packageManager.canRequestPackageInstalls()

    - ACCESS_BACKGROUND_LOCATION: Checked via standard runtime check after fine location.

Question 13

- **Two-step order:** Step 1 requests foreground location (ACCESS_FINE_LOCATION). Step 2 requests ACCESS_BACKGROUND_LOCATION only after foreground is granted.
- **SDK check:** Wrapped in SDK_INT >= Build.VERSION_CODES.Q because background location was introduced in API 29.

Question 14

- **Permission group:** A logical collection of related permissions (e.g., Camera, Contacts).
- **Dialog reduction:** Grouping triggers a single dialog for multiple permissions in the same group.
- **Why not to rely on groups:** Google deprecated group-based auto-granting; permissions are now handled individually.

Question 15

- **What happens:** The permission request fails or returns denied immediately.
- **Declaration difference:** Install-time permissions require only a manifest line. Runtime/Special permissions require manifest declaration *plus* explicit code requests.
- **android:maxSdkVersion:** Used when dropping a permission requirement on newer Android versions (e.g., WRITE_EXTERNAL_STORAGE on API 29+).

Question 16

- **Store filtering (required):** required="true" hides the app on devices without the hardware; required="false" allows installation everywhere.
- **Code check:** With required="false", the app must check packageManager.hasSystemFeature(...) before using it.
- **visibleOn model:** Determines store visibility based on hardware requirements and manifest declarations.

Question 18

- **(1) Auto-reset (Android 11+):** Automatically revokes runtime permissions if an app isn't used for months. Users can turn it off in App Info settings.
- **(2) revokeSelfPermissionOnKill():** Available on API 33+, it explicitly removes runtime permissions your app no longer needs when the process terminates.

Question 19

- **API 23:** Introduced runtime permissions.
- **API 30:** Introduced one-time permissions and auto-reset features.

- **API 33:** Made POST_NOTIFICATIONS a runtime permission and introduced revokeSelfPermissionOnKill().
- **POST_NOTIFICATIONS notice:** Do not request it on versions below API 33 because notifications did not require runtime permissions there.